

Simply Implementing the BFV Scheme

Table of contents

Introduction	2
Warning	2
Context	2
Who could be interested ?	2
What will this crate do ?	3
What will this crate not do ?	3
Concept of Homomorphic encryption	3
RLWE (Ring learning with errors) basics	3
Delving into BFV encryption	6
1 - Mathematical basics	6
2 - Security and performance parameters	7
3 - Key generation	9
The server's public key $a(x)$ generation	9
The client's secret key : $s(x)$	9
The client's public key : $b(x)$	9
Client's public key generation with polynomial sum, multiplication and CBD noise.	10
4 - Plaintext encoding	11
5 - Encryption	13
6 - Decryption	15
7 - Homomorphic operations	19
Sum of two ciphertexts ($C + C$)	19
Sum of a plaintext and a ciphertext ($C + P$)	20
Product of a ciphertext and a plaintext ($C * P$)	22
8 - Noise Management	25
8.1 Noise in a Ciphertext	25
8.2 Decryption Condition	25
8.3 Noise Growth in Our Operations	26
8.4 Working Within Limits	26
8.5 Summary	26
References	27

Introduction

The BFV (Brakerski/Fan-Vercauteren) is a Fully Homomorphic Encryption scheme (FHE), which allows data to be processed while still encrypted. This means we can perform calculations on secure data without needing to decrypt it first. This scheme is practical and useful for keeping data safe during processing, especially in areas like secure communication and privacy-protecting data analysis (Private Information Retrieval, for example). This implementation is for educational purposes.

Warning

First, I want to warn you about the security of this code. This code **isn't** audited, **isn't** intended for production use, **is** vulnerable to timing-based attacks...

So please don't use it seriously. But, you can use it to experiment with BFV encryption, implement an e-voting or PIR with it, for example.

Finally : make everything you imagine with it, but **don't** use it with sensitive data.

Context

BFV implementations already exist, but are often coded in *cpp*¹ and, *fhe.rs*², while being the best FHE crate (including BFV) in Rust for now, is not made for educational purposes.

Who could be interested ?

This crate is designed for people who are searching for a base code for a BFV implementation, for a documented and non-unsafe code, or for people who want to learn about cryptography and especially Learning with Errors & homomorphic cryptography. This implementation isn't designed for experts, nor enterprises who want to use BFV encryption, nor people who just want a safe, secure and audited code.

¹<https://github.com/microsoft/SEAL> for example

²<https://github.com/tlepoint/fhe.rs>

What will this crate do ?

In this project, we'll try to implement a

- RLWE encryption (Key generation, encrypt function, decrypt function)
- FHE operations, such as :
- $C + C$
- $C + P$
- $C * P$

Where C is a ciphertext and P a plaintext.

What will this crate not do ?

This crate will not :

- Implement constant-time equations (be aware of timing attacks !)
- Implement $C * C$, relinearization, nor renormalization.
- Implement SIMD
- Be secured

Concept of Homomorphic encryption

The homomorphic encryption³ allows us to perform computations on encrypted data, without access to the secret key. For example, it allows to do a sum on two ciphertexts :

$$D(E(m_1) \oplus E(m_2)) = m_1 + m_2$$

Where D is the decryption, E is the encryption, $\oplus$ is the operation on ciphertexts.

RLWE (Ring learning with errors) basics

The RLWE is a complicated mathematical problem, considered as computationally hard, even for quantum computers. It makes some modern algorithms secure. For example, CRYSTALS-Kyber⁴ or CRYSTALS-Dilithium⁵ are based on the LWE problem. The RLWE problem is the

³https://en.wikipedia.org/wiki/Homomorphic_encryption

⁴<https://pq-crystals.org/kyber/>

⁵<https://pq-crystals.org/dilithium/>

LWE problem, but in a commutative ring. It makes BFV secure, and it's based on “the assumed intractability of solving certain computational problems if the size of the problem is large enough and the instance of the problem to be solved is chosen randomly⁶” It is built on the arithmetic of polynomials with coefficients from a finite field : the ring. This ring, in BFV encryption, is noted as :

$$\mathbb{Z}_q[X]/(X^n + 1)$$

Where :

$\mathbb{Z}_q$ is a set of integers $\{0, 1, 2, \dots, q - 1\}$ with arithmetic performed modulo q . If q is prime, $\mathbb{Z}_q$ is finite, otherwise, it's a commutative ring. In the code, all the coefficients are reduced modulo q after each operation.

$[X]$ is the polynomial ring construction : $\mathbb{Z}_q[X]$ denotes the set of all polynomials whose coefficients belong to $\mathbb{Z}_q$. (The form of the polynomial is $a(x) = a_0X^0 + a_1X^1 + \dots + a_kX^k$ where $a_i \in \mathbb{Z}_q$)

And we quotient by the ideal generated by $(X^n + 1)$. In the quotient ring, we have the equality $X^n \equiv -1$. It is useful, because any polynomial of degree $\geq n$ is reduced to a polynomial of degree $< n$ by repeatedly substituting X^n with -1 .

Now, let's see what the construction of the problem is :

Let $a_i(x)$ be a random but **known** polynomial with coefficients from all of $\mathbb{Z}_q[X]/(X^n + 1)$. It will be the server's public key.

Let $e_i(x)$ be a random and **unknown** polynomial with coefficients from all of $\mathbb{Z}_q[X]/(X^n + 1)$. We will call this variable the “noise”.

Let $s(x)$ be a **secret** polynomial with small coefficients (bounded by b) in the ring. It will be the secret key of the client.

Let $b_i(x) = (a_i(x) \cdot s(x)) + e_i(x)$ the client's public key.

Under these conditions, recovering the **secret** polynomial $s(x)$ given the pair $(a_i(x), b_i(x))$ is as hard as the degree n of the polynomials and the modulus q are sufficiently large.

The RLWE code used in this implementation is from the crate `simple-`

⁶https://en.wikipedia.org/wiki/Ring_learning_with_errors#Background

ring, which you can find at `/simple-crypto-lab/simple-ring/`⁷. It is recommended to read the documentation⁸ of it before reading this one, because it explains mathematical basics, such as rings, polynomial operations, modular exponentiation or NTT, and it also explains the implementation of it in Rust.

⁷www.github.com/VRAM-RAM/simple-crypto-lab/simple-ring

⁸Documentation of simple-ring

Now, in this section, we'll detail all about the rust code, but also detail mathematical concepts.

Delving into BFV encryption

As you've probably read before, we'll be implementing the BFV (Brakerski-Fan-Vercauteren) encryption scheme in this project. It is one of the most used schemes in the FHE (Fully Homomorphic encryption) domain.

1 - Mathematical basics

As we said, BFV is based on the RLWE (Ring learning with errors) problem. The system's security depends on the difficulty to resolve equations in Euclidean lattices.

BFV doesn't work over individual bits, but over polynomials. It enables parallelism⁹. We define the “polynomials” ring as :

$$R = \mathbb{Z}[X]/(X^n + 1)$$

Where n is a power of 2, every polynomial in R has a maximum degree of $n - 1$, and the polynomial multiplication is done with modulo $(X^n + 1)$, as $X^n \equiv -1$.

For encryption, we work with the quotient ring R_q defined as :

$$R_q = R/(q * R) = \mathbb{Z}_q[X]/(X^n + 1)$$

All quotients are reduced modulo q (a really big unsigned integer), so that they are in the set $\{0, 1, \dots, q - 1\}$.

Finally, for “plaintext” data, we define the ring :

$$R_t = \mathbb{Z}_t[X]/(X^N + 1)$$

Where t is the “message” or “plaintext” modulus (for example $t = 2$ for binary or $t = 256$ for UTF-8 encoding). There is one critical condition : $t \ll q$ (t needs to be **much** less than q).

⁹https://en.wikipedia.org/wiki/Single_instruction,_multiple_data

2 - Security and performance parameters

Before running the code, we need to initialise the parameters :

Parameter	Symbol	Description	Impact
Degree	N	Power of 2	On security and latency
Encryption modulus	q	big unsigned integer	Determine the impact of the noise, and has an impact on the security of encryption, but also latency.
Plaintext modulus	t	small unsigned integer	Determine the values that plaintext coefficients can take.
Scale factor	Δ	$\lfloor \frac{q}{t} \rfloor$	Separate noise and message, and useful to scale plaintext's coefficients into ciphertext's coefficients (we'll see it later).
Bit number	eta	unsigned integer	Determine the distribution of the noise and noise level.

We will define these parameters in our code :

We've decided to create a structure named "BFVParams", which defines the parameters :

```
1 #[derive(Clone)]  
2 pub struct BFV {  
3     pub params: RingParams, //n and q are defined in the  
    RingParams struct, because they belong the mathematical  
    code essentially, while t and eta are cryptographic  
    parameters that don't belong to the mathematical code.  
4     pub t: u64,  
5     pub eta: usize,  
6     pub ntt_precalculated: NTTprecalculated, //For NTT  
    acceleration, we precompute the twiddle factors. (More  
    information in the doc of simple-ring)  
7 }
```

During the code, we'll also use the *Polynomial* struct defined in `simple-ring` :

```
1 pub struct Polynomial {  
2     pub coeffs: Box<[u64]>,  
3 }  
4 impl Polynomial {  
5     pub fn new(c: Vec<u64>) -> Self {  
6         Self { coeffs: c.into_boxed_slice() }  
7     }  
8  
9     pub fn zeros(n: usize) -> Self {  
10        Self { coeffs: vec![0u64; n].into_boxed_slice() }  
11    }  
12 }
```


3 - Key generation

The BFV encryption scheme needs 3 keys : the “server’s” public key, the “client’s” secret key and the “client’s” public key.

The server’s public key $a(x)$ generation

To generate the server’s public key $a(x)$, we will use the function `generate_uniform_polynomial` from `simple-ring`¹⁰, because the coefficients of this key need to be in $[0; q - 1]$ uniformly :

Let c be a coefficient, we have $p(c) = \frac{1}{q}$.

The client’s secret key : $s(x)$

Now, we need a function to generate the secret key of the “client”, $s(x)$. The coefficients of this polynomial are in the coefficient ring, but can only have three values :

$$S(i) \in \{-1, 0, 1\} \bmod q$$

So, to generate the coefficients, we use the function `generate_small_sample` from `simple-ring`, and then we convert the *Sample* into a *Polynomial* in our ring :

```
1 let s_sample = generate_small_sample(&params);  
2 let s = s_sample.to_poly(params.q);
```



The client’s public key : $b(x)$

Finally, to generate the polynomial $b(x)$ which is the client’s public key, we use the formula :

$$b(x) = (a(x) \cdot s(x)) + e(x)$$

Where $b(x)$ is the client’s public key, $s(x)$ is the client’s secret key, $a(x)$ is the server’s public key and $e(x)$ is the random noise.

To compute $b(x)$, we have to use two polynomial operations : the sum,

¹⁰www.github.com/VRAM-RAM/simple-crypto-lab/simple-ring/sampling.rs

and the multiplication, which were implemented in `simple-ring`.

Because the naive multiplication is too slow, instead of using it, we'll use the NTT multiplication.

Finally, we have to generate the noise $e(x)$. It is a polynomial with coefficients generated with the CBD (Centered Binomial Distribution). The generation function we use is - again - a one implemented in `simple-ring`.

Client's public key generation with polynomial sum, multiplication and CBD noise.

Now, we can generate all the keys.

We translate the formula $b(x) = (a(x) \cdot s(x)) + e(x)$ into :

```
1  impl BFV { 🔒 Rust
2      pub fn generate_keys(&self) -> (Polynomial, Polynomial,
3          Polynomial) {
4          let params = &self.params;
5          let ntt_tables = &self.ntt_precalculated;
6          let a = generate_uniform_polynomial(params);
7          let s = generate_small_sample(params);
8          let s = s.to_poly(params.q);
9          let e = generate_cbd_sample(params.n, self.eta);
10         let e = e.to_poly(params.q);
11         let mul = a.mul_ntt(params, ntt_tables, &s);
12         let b = mul.sum(params, &e);
13
14         (a, b, s)
15     }
16 }
```

Note that, in the final version, there will be a function `generate_client_public_key`, a function `generate_server_public_key` and a function `generate_client_secret_key`.

4 - Plaintext encoding

I honestly don't know if doing a part for that is useful, but there is how we implement the plaintext :

First, we create a struct `Plaintext` :

```
1 pub struct Plaintext { Rust
2     pub plain: Polynomial, //The plain is just encoded as a
    polynomial
3     len: usize, //The length of stored data (for example : if
    the plaintext is "Hello", we store 5 bytes with t >=
    256.)
4 }
```

Then, we implement the encoding for the plaintext :

```
1 impl Plaintext { Rust
2     pub fn new(text: &str, bfv: &BFV) -> Self { //Function
    that creates a plaintext, given a text in ASCII or UTF-8
3         let n = bfv.parameters.n;
4
5         let mut coeffs: Vec<u64> = vec![0; n]; //We create
    an empty vector
6
7         let bytes = text.as_bytes(); //We turn the text into
    bytes.
8
9         let len_to_write = bytes.len().min(n); //We deduce
    the len to write
10
11        for i in 0..len_to_write { //We write the coeffs
12            coeffs[i] = bytes[i] as u64;
13        }
14
15        Self {
16            plain: Polynomial::new(coeffs), //We finally
    create the Plaintext
17            len: len_to_write,
18        }
19    }
```

```
20 }
```

It is useful, for debugging or to use BFV on numbers, to have a method that creates the plaintext from raw coefficients :

```
pub fn new_from_coeffs(coeffs: Vec<u64>, bfv: &BFV) -  
1 > Self { //Function to create a plaintext, but  
    directly from a Vec of u64s, not from a String  
2     Self { plain: Polynomial::new(coeffs), len:  
        bfv.parameters.n }  
3 }
```

Rust

If a plaintext is not totally full (for example, with text “Hello” and $n = 4096$, the plaintext contains $\frac{5}{4096}$ coefficients.), it could be useful to “push” coefficient inside of it :

```
1 impl Plaintext {  
2     pub fn push(mut self, to_push: &str) -> Self { //Method  
        to push data into a plaintext  
3  
        let encoded: Vec<u64> =  
4        to_push.as_bytes().iter().map(|&b| b as  
            u64).collect(); //First, we encode the data into a  
            Vec of u64s  
5  
        let remaining = self.plain.coeffs.len() -  
6        self.len; //We calculate the remaining coeffs we can  
            write  
7  
        let len_to_write = encoded.len().min(remaining);  
8  
9        for i in 0..len_to_write { //We write the empty  
            coefficients with pushed ones  
10            self.plain.coeffs[self.len + i] = encoded[i];  
11        }  
12  
13        self.len += len_to_write; //We finally update the  
            len of the plaintext  
14        self  
15    }  
16 }  
17 }
```

Rust

Finally, we need to decode the coefficients, so we implement a function for that :

```
1 pub fn decode(&self) -> String { //Method to decode a plaintext
2     let vec = self.plain.coeffs.to_vec();
3     let vec: Vec<u8> = vec.iter().map(|&b| b as u8).collect();
4     str::from_utf8(&vec).unwrap().to_string() //Doing a .unwrap() is a little brutal, and can make the code panic, so I'll implement Errors.
5 }
```

5 - Encryption

Now, let's encrypt the message.

Let u be a polynomial with small coefficients (such as s).

Let e_1, e_2 be two polynomials, with “CBD noise” as coefficients.

Let $\Delta = \lfloor \frac{q}{t} \rfloor$

Let m be the message we want to cipher.

Let a, b be the keys generated before.

The encryption is noted as :

$$c_0 = ((b \cdot u) + e_1) + (\Delta \cdot m)$$
$$c_1 = (a \cdot u) + e_2$$

Where :

1. $b \cdot u$ and $a \cdot u$ are a polynomial multiplications, which add noise to the system.
2. $+e_1, +e_2$ are polynomial sums, which also adds noise to the system.
3. $\Delta \cdot m$ is a polynomial scaling, which encodes the message.

Then, we return the two ciphertexts $\{c_0, c_1\}$.

For that, we create a struct called `Ciphertext` which will contain c_0 and c_1 :

```
1 #[derive(Debug, Clone)]  
2 pub struct Ciphertext {  
3     pub c0: Polynomial,  
4     pub c1: Polynomial,  
5 }
```

Then, we implement the formula in Rust :

```
1 impl BFV {  
2     pub fn encrypt(&self, message: &BFVPlaintext,  
3         public_key: (&Polynomial, &Polynomial)) -> BFVCiphertext  
4     {  
5         let params = &self.params;  
6         let ntt_tables = &self.ntt_precalculated;  
7         let (a, b) = public_key; //We "extract" the keys  
8         //from the public keypair  
9  
10        let u = generate_small_sample(params); //We generate  
11        //a noise made out of small coefficients  
12        let u = u.to_poly(params.q); //We "pass" it into our  
13        //ring  
14  
15        let e1 = generate_cbd_sample(params.n, self.eta); //  
16        //We generate noise with a Centered Binomial  
17        //Distribution,  
18        let e1 = e1.to_poly(params.q); //Then "pass" it too  
19        //into the ring  
20  
21        let e2 = generate_cbd_sample(params.n, self.eta); //  
22        //Same  
23        let e2 = e2.to_poly(params.q);  
24  
25        let delta = params.q / self.t; //We compute delta  
26    }  
27 }
```

```

18     let bu = b.mul_ntt(params, ntt_tables, &u); //We
compute b * u
19     let delta_m = message.plain.scale(params, delta); //
We encode the plaintext by scaling it
20     let c0_temp = bu.sum(params, &e1); //We continue
computing : b * u + e1
21     let c0 = c0_temp.sum(params, &delta_m); //We finish
computing c0 : (b * u + e1) + delta * m
22
23     let au = a.mul_ntt(params, ntt_tables, &u); //We
compute c1 in two steps. First, we do a * u,
24     let c1 = au.sum(params, &e2); //Then we compute a *
u + e2
25
26     BFVCiphertext { c0, c1 } //We finally return the
Ciphertext
27 }
28 }

```

6 - Decryption

To decrypt the two ciphertexts $\{c_0, c_1\}$, we need to have the secret key s . The decryption formula is :

$$\text{Let } m' = c_0 - c_1 \cdot s$$

We develop :

$$m' = (b \cdot u + e_1 + \Delta \cdot m) - (a \cdot u + e_2) \cdot s$$

We replace b by $a(x) \cdot s(x) + e$:

$$m' = (a \cdot s + e) \cdot u + e_1 + \Delta \cdot m - (a \cdot u + e_2) \cdot s$$

$$m' = (a \cdot s \cdot u) + e \cdot u + e_1 + \Delta \cdot m - (a \cdot u + e_2) \cdot s$$

$$m' = (a \cdot s \cdot u) + e \cdot u + e_1 + \Delta \cdot m - a \cdot s \cdot u - e_2 \cdot s$$

$$m' = e \cdot u + e_1 + (\Delta \cdot m) - e_2 \cdot s$$

And so we have :

$$m' = (\Delta \cdot m) + \text{noise}$$

Because $e \cdot u$, e_1 , $e_2 \cdot s$ are really small.

We want now to extract the message from m' .

$$r = \lfloor \frac{t}{q} \cdot m' \rfloor \bmod t$$

Where m' is the message + the noise, r is the recovered message.

Proof :

We know that :

$$m' \simeq \Delta \cdot m$$

And that :

$$\Delta = \lfloor \frac{q}{t} \rfloor$$

So :

$$r \simeq \lfloor \frac{t}{q} \cdot \Delta \cdot m \rfloor \bmod t$$

And :

$$r \simeq \lfloor \frac{t}{q} \cdot \lfloor \frac{q}{t} \rfloor \cdot m \rfloor \bmod t$$

Finally :

$$r \simeq 1 \cdot m \bmod t$$

We reduce it by t because the message is in $R_t(\mathbb{Z}/(X^n + 1) \bmod t)$. The rounded $\lfloor \cdot \rfloor$ is used to “erase” the noise, so we have the original message. Note that for this to work, the noise needs not to be too strong (it has to be $< \frac{\Delta}{2}$).

Now, let's implement this in Rust :

```
1  pub fn decrypt(&self, ciphertext: &Ciphertext,
2      secret_key: &Polynomial) -> String {
3      let params = &self.parameters;
4      let ntt_tables = &self.precalculated;
5      //First, we compute  $m_{\text{prime}} = c_0 - c_1 * s$ 
6      let cls = Polynomial::polynomial_mul_ntt(params,
7          ntt_tables, &ciphertext.c1, secret_key);
8      let m_prime = Polynomial::polynomial_sub(params,
9          &ciphertext.c0, &cls);
10
11     let mut coeffs = vec![0u64; params.n];
12     let q = params.q as i128;
13     let t = params.t as i128;
14     //Then, for each coefficient, we "erase" the noise. It's
15     //the most delicate part of the decryption. See below for
16     //explanation.
17     for i in 0..params.n {
18         let raw = m_prime.coeffs[i] as i128;
19
20         let val = if raw > q / 2 { raw - q } else { raw };
21
22         let decoded = ((val * t + q / 2).div_euclid(q) % t +
23             t) % t;
24         coeffs[i] = decoded as u64;
25     }
26
27     Plaintext { plain: Polynomial::new(coeffs), len:
28         params.n }.decode()
```

Well, it couldn't be as easy as just applying the mathematical formula. The problem is that if, when decoding, we remain in $\mathbb{Z}_q$, the noise will not be erased, so we have to change of representation. At line 14, we first make m' as integer, because it is, at line 16, centered in $(-q/2; q/2]$.

Why do we center? Because the value m' is computed modulo q , so it satisfies:

$$m' \equiv \widetilde{m}' \bmod q$$

which means that there exists an integer k such that:

$$\widetilde{m}' = m' + k \cdot q$$

Moreover, we have:

$$\widetilde{m}' = \Delta \cdot m + e$$

Without centering, we may use a representative of m' that is far from $\widetilde{m}'$, leading to:

$$r = \lfloor \frac{t}{q} \cdot (\Delta \cdot m + e - k \cdot q) \rfloor$$

which introduces an unwanted term kt after scaling.

With centering, we ensure that we use the representative of m' that is closest to $\widetilde{m}'$. We then compute:

$$r = \lfloor \frac{t}{q} \cdot m' \rfloor \bmod t$$

Since:

$$m' \approx \Delta \cdot m + e = \frac{q}{t} \cdot m + e$$

we obtain:

$$r = \lfloor m + \left(\frac{t}{q}\right) \cdot e \rfloor \bmod t$$

If the noise e is small enough, the rounding removes it and we recover:

$$r = m \bmod t$$

Hence:

$$r = m \in R_t$$

7 - Homomorphic operations

The core of BFV is the operations we can compute on ciphertexts. There are 4 operations (well in fact many more, like *relinearization*, *normalisation*, *modulo switching*... but these operations won't be used nor implemented here). The main operations in BFV are :

1. The sum of a plaintext and a ciphertext $C + P$,
2. The sum of two ciphertexts $C + C$,
3. The product of one ciphertext and one plaintext $C * P$,
4. The product of two ciphertexts $C * C$

Implementing these operations isn't difficult, except for the fourth one, the product of two ciphertexts, because, during the product of two ciphertexts, the degree of the polynomials changes, so we have to introduce *relinearization*, but also the noise grows exponentially, so we have to introduce *normalization*. But, even with these two operations, without implementing the *RNS*¹¹ (Residue Number System), implementing the product is too difficult, and technically impossible (the performance is affected if *RNS* isn't implemented).

If you're interested in it, please see the documentation at the end of the document.

Sum of two ciphertexts ($C + C$)

In the BFV FHE scheme, the most simple operation to implement is the sum of two ciphertexts, because it's just a polynomial sum computed on the two ciphertexts. We implement it as a method of BFV :

```
1  impl BFV { 🔒 Rust
2      pub fn sum_ciphertexts(&self, first_cipher: Ciphertext,
3                             second_cipher: Ciphertext) -> Ciphertext {
4          let params = &self.parameters;
5          let new_c0 = first_cipher.c0.sum(params,
6          &second_cipher.c0);
7          let new_c1 = first_cipher.c1.sum(params,
8          &second_cipher.c1);
9          Ciphertext { c0: new_c0, c1: new_c1 }
10     }
11 }
```

¹¹https://en.wikipedia.org/wiki/Residue_number_system

Sum of a plaintext and a ciphertext ($C + P$)

The sum of a plaintext and a ciphertext is not as simple as the sum of two ciphertexts, because :

1. The plaintext is not scaled like the message m which was scaled by Δ .
2. The plaintext is compound by a polynomial, but the ciphertext is compound by two polynomials.

First, we scale the plaintext :

$$p' = p \cdot \Delta$$

Then, we do :

$$c_0' = c_0 + p'$$

$$c_1' = c_1$$

And the sum is done !

Proof :

Let m be the initial message, p be a plaintext we sum, b, a, s the keys and u, e the noises :

$$c_0 = ((b \cdot u) + e_1) + (\Delta \cdot m)$$

$$c_1 = (a \cdot u) + e_2$$

$$c_0' = c_0 + p \cdot \Delta$$

$$c_1' = c_1$$

So, in decryption :

$$\text{Let } m' = c_0' - c_1' \cdot s$$

We develop :

$$m' = (b \cdot u + e_1 + \Delta \cdot m) + p \cdot \Delta - (a \cdot u + e_2) \cdot s$$

We replace b by $a(x) \cdot s(x) + e$:

$$m' = (a \cdot s + e) \cdot u + e_1 + \Delta \cdot m + p \cdot \Delta - (a \cdot u + e_2) \cdot s$$

$$m' = (a \cdot s \cdot u) + e \cdot u + e_1 + \Delta \cdot m + p \cdot \Delta - (a \cdot u + e_2) \cdot s$$

$$m' = (a \cdot s \cdot u) + e \cdot u + e_1 + \Delta \cdot m + p \cdot \Delta - a \cdot s \cdot u - e_2 \cdot s$$

$$m' = e \cdot u + e_1 + (\Delta \cdot m) + (p \cdot \Delta) - e_2 \cdot s$$

And so we have :

$$m' = (\Delta \cdot m) + (\Delta \cdot p) + \text{noise}$$

Finally :

$$r = \lfloor \frac{t}{q} \cdot m' \rfloor \bmod t$$

$$r = \lfloor \frac{t}{q} \cdot [(\Delta \cdot m) + (\Delta \cdot p)] + \text{noise} \rfloor \bmod t$$

$$r = \lfloor \frac{t}{q} \cdot \left[\left(\frac{q}{t} \cdot m \right) + \left(\frac{q}{t} \cdot p \right) \right] + \text{noise} \rfloor \bmod t$$

$$r = \lfloor \frac{t}{q} \cdot \left[\left(\frac{q}{t} \cdot m \right) + \left(\frac{q}{t} \cdot p \right) \right] + \text{noise} \rfloor \bmod t$$

$$r = \lfloor m + p + \text{noise} \rfloor \bmod t$$

$$r = m + p \bmod t$$

We can now implement it in our code :

```
1  impl BFV { 🔒 Rust
2      pub fn sum_ciphertext_and_plaintext(&self, ciphertext:
3          &Ciphertext, plaintext: &Plaintext) -> Ciphertext { //
4              Function to sum a ciphertext and a plaintext
5              let params = &self.parameters;
6              let delta = params.q / params.t;
7              let delta_m = plaintext.plain.scale(params,
8                  delta); //We first need to scale the plaintext
9                  coeffs with delta
10             let new_c0 = ciphertext.c0.sum(params, &delta_m); //
11             Then we can sum, but only on c0, not on c1
12             let new_c1 = ciphertext.c1.clone();
13
14             Ciphertext { c0: new_c0, c1: new_c1 }
15     }
```

Product of a ciphertext and a plaintext ($C * P$)

The product of a ciphertext and a plaintext is the last operation we'll implement in this implementation. It is as simple as $C + C$, because the formula of this product is :

$$c_0' = c_0 \cdot p$$

$$c_1' = c_1 \cdot p$$

Here, we don't have to scale nor to apply it on only one coefficient, it's really intuitive.

Proof :

Let m be the initial message, p be a plaintext we multiply, b, a, s the keys and u, e the noises :

$$c_0 = ((b \cdot u) + e_1) + (\Delta \cdot m)$$

$$c_1 = (a \cdot u) + e_2$$

$$c_0' = c_0 \cdot p$$

$$c_1' = c_1 \cdot p$$

So, in decryption :

$$\text{Let } m' = c_0' - c_1' \cdot s$$

We develop :

$$m' = (b \cdot u + e_1 + \Delta \cdot m) \cdot p - (a \cdot u + e_2) \cdot s \cdot p$$

We replace b by $a(x) \cdot s(x) + e$:

$$m' = (a \cdot s + e) \cdot u \cdot p + e_1 \cdot p + \Delta \cdot m \cdot p - (a \cdot u + e_2) \cdot s \cdot p$$

$$m' = (a \cdot s \cdot u \cdot p) + e \cdot u \cdot p + e_1 \cdot p + \Delta \cdot m \cdot p - (a \cdot u + e_2) \cdot s \cdot p$$

$$m' = (a \cdot s \cdot u \cdot p) + e \cdot u \cdot p + e_1 \cdot p + \Delta \cdot m \cdot p - a \cdot s \cdot u \cdot p - e_2 \cdot s \cdot p$$

$$m' = e \cdot u \cdot p + e_1 \cdot p + (\Delta \cdot m \cdot p) - e_2 \cdot s \cdot p$$

And so we have :

$$m' = (\Delta \cdot m \cdot p) + \text{noise}$$

Finally :

$$r = \lfloor \frac{t}{q} \cdot m' \rfloor \bmod t$$

$$r = \lfloor \frac{t}{q} \cdot (\Delta \cdot m \cdot p) + \text{noise} \rfloor \bmod t$$

$$r = \lfloor \frac{t}{q} \cdot (\frac{q}{t} \cdot m \cdot p) + \text{noise} \rfloor \bmod t$$

$$r = \lfloor m \cdot p + \text{noise} \rfloor \bmod t$$

$$r = m \cdot p \bmod t$$

As you can see, in this operation, the noise is scaled by p three times, so it becomes louder, while in the sum the noise wasn't affected by the sum. It means that, after many products, the noise can be too loud so that the message doesn't decrypt correctly.

Now, let's implement it in our code :

```
1  impl BFV { Rust
2      pub fn mul_ciphertext_and_plaintext(&self, ciphertext:
          &Ciphertext, plaintext: &Plaintext) -> Ciphertext {
3          let params = &self.parameters;
4          let ntt_tables = &self.precalculated;
5          let new_c0 = ciphertext.c0.mul_ntt(params,
          ntt_tables, &plaintext.plain);
6          let new_c1 = ciphertext.c1.mul_ntt(params,
          ntt_tables, &plaintext.plain);
7
8          Ciphertext { c0: new_c0, c1: new_c1 }
9      }
10 }
```


8 - Noise Management

In BFV, noise is intentional: it provides security via the RLWE assumption. But it is also a constraint : if noise grows too large, decryption fails.

This section focuses on how noise behaves in the operations we implemented ($C + C$, $C + P$, $C \times P$), without advanced techniques like RNS or bootstrapping.

8.1 Noise in a Ciphertext

After encryption, a ciphertext (c_0, c_1) satisfies:

$$[c_0 + c_1 \times s]_q = \Delta \times m + e_t \pmod{q}$$

Where $e_t = e_1 + e_2 \times s + e \times u$ combines:

e_1, e_2 : CBD-sampled noise polynomials
e : small error from the public key $b = -a \times s + e$
u : small random polynomial used during encryption

All components are small, so e_t starts small.

8.2 Decryption Condition

Decryption computes:

$$m' = \lfloor \left(\frac{t}{q} \right) \times [c_0 + c_1 \times s]_q \rfloor \pmod{t}$$

This recovers m correctly **only if**:

$$\|e_t\|_\infty < \frac{\Delta}{2}$$

where $\Delta = \lfloor \frac{q}{t} \rfloor$ and $\|\cdot\|_\infty$ is the maximum absolute coefficient.

If noise exceeds $\frac{\Delta}{2}$, rounding crosses a boundary and decryption returns a wrong value, silently.

8.3 Noise Growth in Our Operations

Operation	Noise growth	Rough limit
$C + C$	Additive: $\ e_1\ + \ e_2\ $	50–100 additions
$C + P$	None: $\ e\ $ unchanged	Unlimited
$C \times P$	Multiplicative: $\ p\ \times \ e\ $	5–10 multiplications

These limits assume typical parameters ($q \approx 2^{19}$, $t = 256$, $\eta = 8$). Adjust based on your chosen q , t , and η .

8.4 Working Within Limits

Since our implementation does not include RNS, modulus switching, or bootstrapping:

- **Plan depth ahead:** Count multiplications; keep sequential $C \times P$ operations low.
- **Prefer small plaintexts:** Use $t = 2$ (binary) for heavy computations to limit noise scaling.
- **Monitor during development:** Optionally compute $\|[c_0 + c_1 \times s]_q - \Delta \times m\|_\infty$ to estimate noise (educational use only), with the function `estimate_noise`.
- **Re-encrypt if possible:** If you hold the secret key, decrypting and re-encrypting resets noise.

8.5 Summary

Key takeaways:

Noise is required for security but must stay below $\frac{\Delta}{2}$ for correct decryption.

Addition grows noise slowly; multiplication grows it faster.

Without RNS or bootstrapping, computation depth is limited—plan accordingly.

For production use, consider libraries with advanced noise management (SEAL, OpenFHE).

References

- [1] : Fan, Junfeng and Frederik Vercauteren. “Somewhat Practical Fully Homomorphic Encryption.” IACR Cryptol. ePrint Arch. 2012 (2012): 144. (<https://eprint.iacr.org/2012/144.pdf>)
- [2] : Mali, B. Raut. “Generalized BGV, BFV, and CKKS for Homomorphic Encryption over Matrix Rings.” IACR Cryptol. ePrint Arch. 2025 (2025): 972. (<https://eprint.iacr.org/2025/972.pdf>)
- [3] : Homomorphic encryption on Wikipedia (https://en.wikipedia.org/wiki/Homomorphic_encryption)
- [4] : Özcan, Ali & Ayduman, Can & Türkoğlu, Enes & Savaş, Erkey. (2023). Homomorphic Encryption on GPU. IEEE Access. PP. 1-1. 10.1109/ACCESS.2023.3265583. (<https://eprint.iacr.org/2022/1222.pdf>)
- [5] : Inferati Inc., “Introduction to the BFV FHE Scheme.” [Online]. Available: PDF document. (<https://faculty.kfupm.edu.sa/coe/mfelemban/SEC595/References/Introduction%20to%20the%20BFV%20FHE%20Scheme.pdf>)
- [6] : Linus Tang, “BFV Homomorphic Encryption: Part 1,” Massachusetts Institute of Technology. [Online]. Available: https://www.mit.edu/~linust/files/BFV_Homomorphic_Encryption_Part_1.pdf. [Accessed: May 16, 2026].